Universidad de los Andes
Facultad de Ingeniería y Ciencias Aplicadas
Semestre 2026-2
Arquitectura de Computadores
Profesor: Jorge Gómez Mir

Proyecto 3

El proyecto considera una entrega en GitHub y una defensa presencial:

- **Fecha de entrega en GitHub: 4/11 - 10:30am**.
- **Defensa en clases: 4/11 - 10:30am**.

Utilizar los mismos grupos del proyecto anterior. El punto de partida es **el repositorio entregado por su grupo en el Proyecto 2** (RTL modificado, `game.s` y assembler), no el repositorio base.

1. Objetivo

Llevar el Pochoco SoC, con el juego de reflejos del Proyecto 2, desde RTL hasta un **GDSII**, usando el flujo abierto **LibreLane** y el PDK **GF180MCU**. El grupo deberá adaptar el diseño de FPGA a ASIC, tomar decisiones de arquitectura sobre la memoria, explorar el espacio de diseño y justificar cada decisión con datos de sus propias corridas.

El resultado será un **macro digital implementado con celdas estándar**, no un chip completo listo para encapsular. Los puertos del diseño representan conexiones lógicas que, en un chip completo, se conectarían a pads de I/O con sus respectivos circuitos de protección y manejo de carga.

Para asegurar que los resultados sean reproducibles, el README deberá registrar la versión de LibreLane, la revisión del PDK y la biblioteca de celdas estándar usadas en la corrida final.

2. Parámetros del diseño

Todos los grupos trabajan con los mismos parámetros para su corrida final:

Parámetro	**Valor**
Período de reloj objetivo T (`CLOCK_PERIOD`)	100 ns
Utilización de core objetivo U (`FP_CORE_UTIL`)	40 %

La corrida final se considerará cerrada cuando LibreLane termine el flujo, el diseño tenga DRC y LVS limpios, no presente violaciones de antena sin justificar y cumpla setup y hold en los *corners* exigidos. Para timing, se requiere WNS ≥ 0 y TNS $= 0$ tanto para setup como para hold.

3. Actividades

3.1. Del RTL de FPGA al RTL de ASIC

Cree un módulo superior `asic_top.v` que instancie el SoC sin elementos propios de la FPGA. Sus puertos deben corresponder a las funciones lógicas del juego: reloj, reset, LEDs, botones, displays de 7 segmentos y SPI. Estos puertos son terminales del macro y no pads físicos capaces de manejar directamente los dispositivos externos.

Identifique **cada construcción de su RTL del Proyecto 2 que dependa de la FPGA**: inicialización con `$readmemh`, bloques `initial`, inferencia de BRAM, supuestos sobre el reloj o el reset, entre otras. Para cada una, adjunte el *warning* o error de LibreLane que la evidenció (archivo de log y línea) y explique cómo la reemplazó. Si alguna no produjo ningún mensaje del flujo, explique por qué seguía siendo un problema para una implementación ASIC.

3.2. Memoria y programa del juego

En silicio no existe un archivo `game.hex` que se cargue automáticamente al encender. El grupo deberá **implementar y comparar estas dos estrategias** para la memoria de instrucciones y datos:

1. una ROM combinacional generada automáticamente por el assembler a partir de `game.s`, más una RAM de datos reducida;
2. una memoria unificada implementada con flip-flops, dimensionada según el uso real del juego.

Para mantener un alcance comparable entre grupos, se recomienda un máximo de 256 palabras de instrucciones y 64 palabras de datos. Si el programa requiere superar uno de estos límites, el grupo deberá justificarlo antes de ejecutar la comparación final. La implementación de un *bootloader* por SPI es opcional y no reemplaza las dos estrategias exigidas.

Para cada estrategia, reporte área de celdas, número de flip-flops, utilización y frecuencia máxima obtenidas de sus corridas. Indique cuántas palabras de instrucciones y de datos usa realmente su juego y explique cómo obtuvo esas cantidades.

3.3. Síntesis

A partir del reporte de síntesis de su corrida final:

- indique las cinco celdas más usadas y qué parte del diseño explica la cantidad de cada una;
- determine qué módulo aporta más área y qué fracción corresponde al **contador de ciclos agregado en el Proyecto 2**;
- compare el resultado con lo que esperaba antes de sintetizar. Si no coincide, explique la diferencia.

El grupo deberá conservar nombres de módulos e instancias que permitan identificar, al menos, el register file, la ALU y el contador de ciclos. Si la síntesis aplana u optimiza la jerarquía, deberá obtener las áreas mediante estadísticas jerárquicas previas al aplanamiento o mediante la suma de las celdas asociadas a cada instancia, y documentar el método utilizado.

3.4. Timing

- Encuentre el menor período para el cual el diseño cierra timing, barriendo `CLOCK_PERIOD` en al menos tres puntos, incluyendo T. Presente una tabla con WNS y TNS de setup y hold para el *corner* típico y el más lento. Considere que una corrida cierra timing cuando cumple los criterios definidos en la sección anterior.
- Para la ruta crítica, indique la celda de inicio, la celda de fin, el módulo y la función microarquitectónica de Espino a la que corresponde: fetch, decode, ejecución, acceso a memoria o write-back.
- Proponga un cambio al RTL que pueda acortar esa ruta, impleméntelo y muestre el nuevo slack y su efecto en el área. Si el cambio no mejoró el resultado, explique por qué.
- Si el macro funcionara con período T, ¿qué constantes de `game.s` deben cambiar para que la espera siga siendo de tres segundos y el tiempo siga mostrándose en décimas de segundo? Muestre el cálculo y regenere el programa.

La frecuencia máxima se reportará a partir del menor período que cierre timing en todos los *corners* exigidos. El grupo deberá indicar que cada valor de `CLOCK_PERIOD` puede provocar una nueva optimización del circuito y, por tanto, modificar área, buffering y placement.

3.5. Floorplan, red de alimentación y placement

- Explore al menos tres valores de `FP_CORE_UTIL` o tres dimensiones de `DIE_AREA`, y determine el área mínima con la que el diseño completa el flujo según los criterios establecidos. Grafique la utilización frente al número de violaciones o a una métrica de congestión reportada por el flujo.
- Justifique la ubicación de los terminales del macro en el borde del área de diseño según su función. Distinga estos terminales de los pads de I/O que necesitaría un chip completo.
- Usando el dato visto en clases (US$23.000 por un área de 1 mm^2), estime el costo equivalente de su diseño **suponiendo, solo para este ejercicio, que el costo escala linealmente con el área**. Explique las limitaciones de esta aproximación y discuta qué decisión de las secciones anteriores tuvo mayor impacto en el área y el costo estimado.

3.6. Signoff y layout

- Reporte los resultados finales de DRC, LVS y antena. Para DRC y LVS se exige una corrida limpia. Toda violación de antena debe corregirse o justificarse con evidencia del reporte y una explicación de su impacto. Si durante el desarrollo apareció alguna violación, describa una, muestre dónde estaba en el layout y explique cómo la resolvió.
- En KLayout u OpenROAD GUI, **marque en el layout** dónde quedaron el register file, la ALU y el contador de ciclos. Identifique las celdas mediante los nombres jerárquicos conservados o mediante el método de trazabilidad documentado en la sección de síntesis. Adjunte las capturas anotadas.

3.7. Simulación funcional post-síntesis

Simule el netlist post-síntesis, sin retardos anotados, con su programa del juego y un testbench que presione los botones en ciclos de reloj definidos. Muestre que:

- una ronda correcta produce el mismo número de ciclos medidos que la simulación RTL cuando los estímulos se aplican en los mismos ciclos;
- una pulsación incorrecta reinicia la ronda actual.

Puede acortar la espera de tres segundos para la simulación, siempre que documente el cambio y utilice la misma configuración en ambas simulaciones. Una simulación post-PnR con retardos SDF es opcional.

4. Defensa presencial

Durante la defensa se harán preguntas sobre cualquier parte del informe, del RTL y del assembler. Todos los integrantes deben poder responder.

El README debe indicar cómo instalar LibreLane, cómo ejecutar la corrida final, cómo regenerar el programa desde `game.s` y las versiones exactas de las herramientas, el PDK y la biblioteca utilizadas.

5. Entrega

En la entrega se considerarán los siguientes ítems:

Ítem	**Puntaje**
Del RTL de FPGA al RTL de ASIC (PDF)	0,5
Memoria, programa del juego y extensión del assembler (código + PDF)	1,0
Síntesis y trazabilidad jerárquica (PDF)	0,4
Timing: barrido, ruta crítica, optimización y constantes del juego (PDF)	1,0
Floorplan, alimentación, placement y costo (PDF)	0,5
Signoff y layout anotado (PDF)	0,5
Simulación funcional post-síntesis (código + PDF)	0,6
Defensa presencial y preguntas	1,5
Total	**6,0**

Referencias: https://librelane.readthedocs.io/en/stable/ https://gf180mcu-pdk.readthedocs.io/en/latest/ https://openroad.readthedocs.io/en/latest/